Multi-Agent Intelligence Research System - Design Report

Version: 1.0
Date: July 24, 2025

1. Introduction

This design report provides an overview of the Multi-Agent Intelligence Research System, a Python-based framework for automated competitive intelligence and product update reporting. The system orchestrates specialized agents to perform search, summarization, and verification tasks, culminating in the generation of reliable intelligence reports.

2. Objectives

- Automate information gathering from diverse sources (web, news, blogs, PDFs)
- Synthesize large volumes of text into concise summaries
- Validate content accuracy via zero-shot classification and LLM verification
- Offer both command-line and interactive web interfaces

3. High-Level Architecture

3.1 Entry Points

- CLI (main.py): Parses arguments, sets up logging and memory, invokes CoordinatorAgent.
- Web UI (app.py): Renders Streamlit interface, captures user inputs, invokes CoordinatorAgent.

3.2 Configuration

- config.py loads environment variables via python-dotenv.
- Global parameters (timeouts, retry limits, categories) can be overridden in .env.

4. Component Design

4.1 CoordinatorAgent

- Role: Orchestrates the research pipeline.
- Methods:
  - execute(query, category): Drives phase transitions, collects results, saves report.
- Dependencies: utils.logger, utils.memory.SimpleMemory, agents.*.

4.2 SearchAgent + SearchTool

- Role: Aggregate heterogeneous search results.
- SearchTool methods:
  - _search_general(): DuckDuckGo (DDGS)
  - _search_news(): HTTP requests + BeautifulSoup domain filtering
  - _search_blogs(): Asynchronous crawling via Playwright + bs4
  - _search_pdf(): HTTP + PyPDF2 parsing

4.3 SummarizerAgent

- Role: Condense text into summaries.
- Uses Hugging Face pipeline('summarization', model='facebook/bart-large-cnn').
- Splits long articles via regular expressions.

4.4 VerifierAgent

- Role: Validate summaries for accuracy.
- Initializes LLM pipeline distilgpt2 for text generation.
- Utilizes LangChain ZeroShotClassificationChain to perform zero-shot labeling.

5. Data Flow

1. User submits query → CoordinatorAgent
2. SearchAgent collects raw documents → list of dicts
3. SummarizerAgent generates summaries → list of texts
4. VerifierAgent classifies summaries → labels/score
5. CoordinatorAgent aggregates and writes JSON report

6. Technical Stack

- Language: Python 3.8+
- Libraries: requests, PyPDF2, duckduckgo_search, playwright, beautifulsoup4, transformers, torch, python-dotenv, streamlit
- LLMs: facebook/bart-large-cnn (summarization), distilgpt2 (verification)
- Frameworks: Streamlit (UI), Playwright (browser automation)
- Utilities: LangChain for zero-shot classification, custom logging and memory modules

7. Known Limitations & Future Work

- No vector embeddings for similarity search
- Limited error handling; enhance retry logic and backoff strategies
- Expand to authenticated or paid APIs for richer data

8. Challenges

1. Critical Web Scraping Failure (NotImplementedError): The core issue preventing the system from working is the failure of the playwright library to launch a browser process for content extraction. This points to a fundamental problem with playwright's execution in a specific environment (Python 3.12 on Windows).
2. Deprecated Dependency Warning: The duckduckgo_search package is deprecated.
3. PyTorch Warnings: Warnings like Tried to instantiate class '__path__._path' appear but seem unrelated to the main scraping failure.
4. LLM Context Window Limits : Although not directly causing the current failure, the system's design must inherently deal with the limited context window size of the BART summarization model. If content were successfully extracted, very long web pages or a large number of results might need to be chunked or filtered before summarization to fit within this limit. The current failure prevents this aspect from being tested, but it's a known constraint for LLM-based summarization.

9. Performance Considerations

- Caching: In-memory caching reduces duplicate searches and model calls, controlled by utils.memory.
- Asynchronous Crawling: Using Playwright's async API accelerates blog scraping while bounding concurrency.
- Model Offloading: Option to offload heavy summarization and verification to GPUs (future).
- Rate Limiting: Respect external site rate policies via throttling logic in SearchTool.

10. Future Enhancements

- Integrate vector embeddings (e.g., FAISS, Pinecone) for semantic search and similarity ranking.
- Expand reporting formats: CSV, HTML dashboards, PDF exports.
- Introduce authentication modules for paid APIs (OAuth, API keys).

11. Key Design Patterns

- Agent Pattern: Encapsulates logic in self-contained agent classes (BaseAgent abstraction).
- Pipeline Pattern: Sequential processing stages orchestrated by CoordinatorAgent.
- Factory Method: Hugging Face pipeline() factory instantiates models.
- Strategy Pattern: Swappable search strategies in SearchTool for different domains and formats.

- Runs Totally Local without heavy API fees for LLM's.

12. Supporting Infrastructure

- Logging: Configurable via utils.logger and environment variables- Configuration Management: Centralized in config.py with overrides via .env.
- Memory & Caching: utils.memory.SimpleMemory for storing interim results and interactions.
- Environment Setup: Managed dependencies via requirements.txt and Playwright browsers installation.

Conclusion

The Multi-Agent Competitive Intelligence System successfully demonstrates the power of coordinated AI agents for complex information processing tasks while operating entirely locally.

The system achieves its primary objectives of simulated intelligence gathering, structured information extraction, and reliable report generation. The design choices prioritize maintainability, extensibility, and ease of use while demonstrating the inherent capabilities of multi-agent coordination.